Project Requirements & Technical Roadmap: Clinically Aware Healthcare Chatbot

Student Team: Advait Dintakurti, Venya Velmurugan, Amish Goyal, Keshav Goel, Vikesh Kansal

January 29, 2026

1 Executive Summary

The objective of this project is to develop a privacy-preserving, "clinically aware" chatbot for **NanoCare Health Services**. Unlike generic Large Language Models (LLMs), this system will synthesize patient-specific medical history to provide personalized diagnostic support. **The initial deliverable will be a functional Python-based prototype** designed to demonstrate the core RAG capabilities, capable of future API integration with the client's existing Ruby on Rails infrastructure.

2 Resource Requirements (Client Deliverables)

To proceed with the development phase effectively, the Student Team requests the following resources:

- **Anonymized Data Access:** A comprehensive dataset of anonymized EMR records in JSON format to train the ingestion pipeline.
- **API Specifications (Optional):** If available, API docs from the existing Rails backend to guide our future integration strategy (though the prototype will be standalone).
- **Compute Specifications:** Details regarding deployment hardware (GPU VRAM) for the eventual local model.
- **API Credentials (Phase 1):** Access keys for secure cloud LLM providers (e.g., Azure OpenAI) for initial prototyping.

3 Proof of Concept (PoC) Technical Rationale

Why this specific architecture solves the "Clinical Awareness" problem:

3.1 1. Solving the "Hallucination" Problem (Grounding)

Challenge: Generic LLMs often invent facts when asked about specific patients.
Solution: Our Retrieval-Augmented Generation (RAG) approach "grounds" the model. By forcing the LLM to use *only* the retrieved EMR data as its source of truth, we mathematically minimize the probability of hallucination.

3.2 2. Solving the "Context Window" Limit

Challenge: A patient's medical history over 10 years is too large to paste into a single AI prompt.
Solution: The **Vector Database** acts as a semantic filter. It allows us to pinpoint and retrieve only the specific 1% of the medical history relevant to the *current* question (e.g., pulling only "Cardiac Reports" when the user asks about chest pain).

4 Proposed Technical Architecture

4.1 1. System Workflow Diagram

The following architecture illustrates the end-to-end data flow from the Client UI to our Private LLM Inference engine.

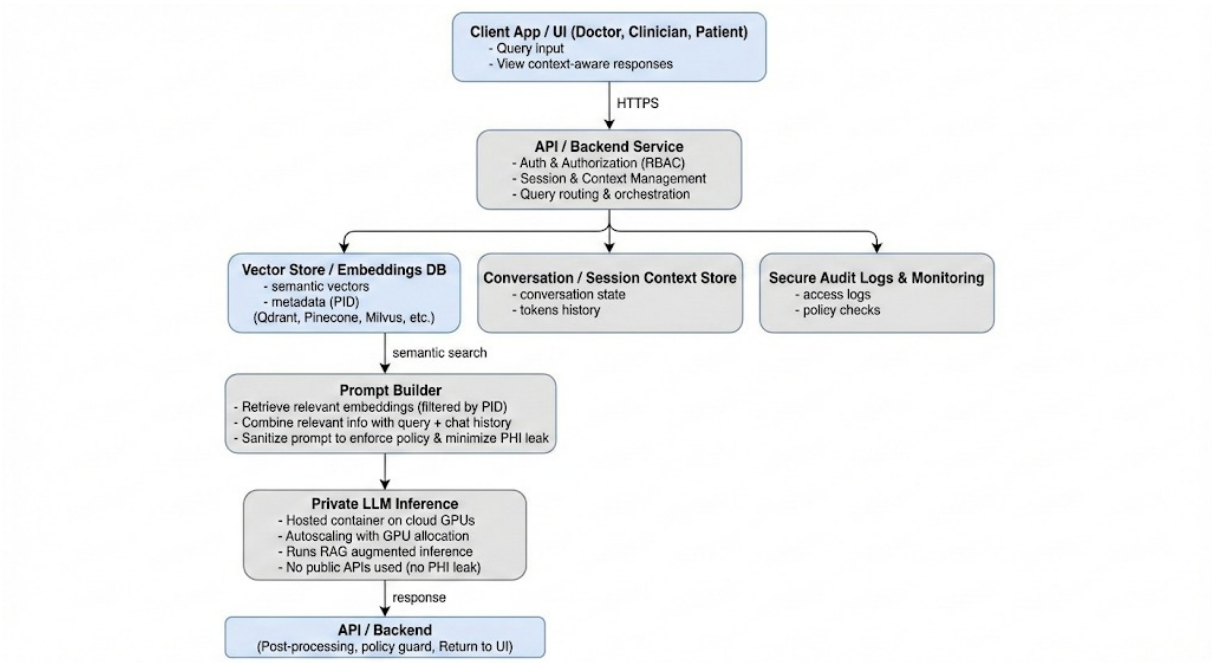

Figure 1: Proposed System Architecture Data Flow

4.2 2. Prototype Architecture (Python Stack)

To ensure rapid development and proof-of-capability, the Prototype will be built entirely within the Python ecosystem:

- **Core Microservice (FastAPI):** The logic layer handling RAG, Context Retrieval, and Sanitization will be built as a high-performance Python microservice.
- **Prototype UI (Streamlit/Python):** For the demonstration phase, we will deploy a lightweight Python-based user interface. This allows the client to test the "Clinical Awareness" without requiring immediate changes to their production Ruby on Rails app.
- **Future Integration:** Once validated, the Core Microservice exposes standard REST APIs (JSON) that can be easily consumed by the client's existing Ruby on Rails backend.

4.3 3. The RAG Processing Pipeline (Detailed)

Step A: Context Retrieval
The system queries the Vector Store for records matching the specific medical intent.

Step B: The Prompt Builder & Sanitization
We construct a secure system prompt combining: (1) User Query, (2) Retrieved Medical Evidence, and (3) Chat History. A **Sanitization Layer** runs here to redact any sensitive PII.

Step C: Private LLM Inference
The prompt is processed by the model (Cloud API in Phase 1 $\rightarrow$ Local LLM in Phase 2).

5 Project Roadmap (12 Weeks)

Timeline	Phase	Key Deliverables
Weeks 1-2	**Discovery & Data Analysis**	• Analyze EMR JSON structure. • Define Vector DB schema. • Finalize Python tech stack.
Weeks 3-4	**Core Infrastructure Setup**	• Setup Vector Database (Qdrant/Milvus). • Build Python Ingestion Pipeline for JSON records. • *Milestone: Efficient retrieval of patient history.*
Weeks 5-7	**Prototype Development (Phase 1)**	• Implement Multi-stage RAG pipeline (FastAPI). • **Develop Python Demo UI (Streamlit).** • Testing with Cloud APIs.
Weeks 8-10	**Local LLM Transition (Phase 2)**	• Evaluate open-source models (BioMistral, MedLlama). • Fine-tuning/Quantization for local hardware. • Replace Cloud API with Local Container.
Weeks 11-12	**Validation & Handover**	• Red Teaming for safety. • Final Documentation & Deployment.